

```
In [1]: # Cell 1: Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re

%matplotlib inline
sns.set(style="whitegrid")

# Cell 2: Read data
transaction = pd.read_csv("QVI_transaction_data.csv")
behaviour = pd.read_csv("QVI_purchase_behaviour.csv")

print("Transactions shape:", transaction.shape)
print("Behaviour shape:", behaviour.shape)

# Cell 3: Quick look
display(transaction.head())
display(behaviour.head())

transaction.info()
behaviour.info()
```

Transactions shape: (264836, 8)
Behaviour shape: (72637, 3)

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_
0	43390	1	1000	1	5	Chip Natural Compny SeaSalt175g	
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	
3	43329	2	2373	974	69	Thinly S/Cream&Onion Smiths Chip 175g	
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

```
<class 'pandas.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   DATE                  264836 non-null int64
1   STORE_NBR             264836 non-null int64
2   LYLTY_CARD_NBR        264836 non-null int64
3   TXN_ID                264836 non-null int64
4   PROD_NBR              264836 non-null int64
5   PROD_NAME             264836 non-null str
6   PROD_QTY              264836 non-null int64
7   TOT_SALES             264836 non-null float64
dtypes: float64(1), int64(6), str(1)
memory usage: 16.2 MB
<class 'pandas.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
0   LYLTY_CARD_NBR        72637 non-null int64
1   LIFESTAGE              72637 non-null str
2   PREMIUM_CUSTOMER      72637 non-null str
dtypes: int64(1), str(2)
memory usage: 1.7 MB
```

```
In [2]: # Cell 4: Fix DATE (Excel integer date → real date)
transaction['DATE'] = pd.to_datetime(transaction['DATE'], unit='D', origin='1900-01-01')

# Corrected line (removed the old argument)
print(transaction['DATE'].describe())
```

```
count          264836
mean    2018-12-30 00:52:12
min      2018-07-01 00:00:00
25%      2018-09-30 00:00:00
50%      2018-12-30 00:00:00
75%      2019-03-31 00:00:00
max       2019-06-30 00:00:00
Name: DATE, dtype: object
```

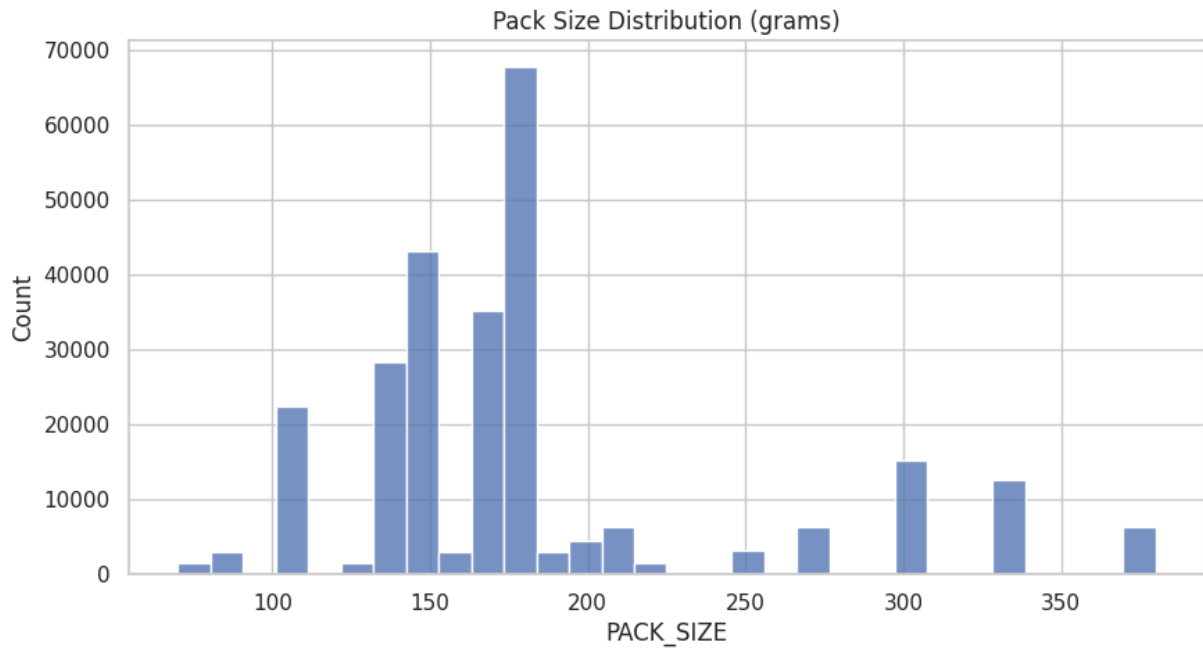
```
In [3]: # Cell 5: Examine PROD_NAME – most frequent products
print(transaction['PROD_NAME'].value_counts().head(20))

# Cell 6: Extract pack size (using regex)
```

```
transaction['PACK_SIZE'] = transaction['PROD_NAME'].str.extract(r'(\d+)g?',
print(transaction['PACK_SIZE'].value_counts().sort_index())

# Visualize pack size distribution
plt.figure(figsize=(10,5))
sns.histplot(transaction['PACK_SIZE'], bins=30)
plt.title("Pack Size Distribution (grams)")
plt.show()
```

```
PROD_NAME
Kettle Mozzarella Basil & Pesto 175g 3304
Kettle Tortilla ChpsHny&Jlpno Chili 150g 3296
Cobs Popd Swt/Chlli &Sr/Cream Chips 110g 3269
Tyrrells Crisps Ched & Chives 165g 3268
Cobs Popd Sea Salt Chips 110g 3265
Kettle 135g Swt Pot Sea Salt 3257
Tostitos Splash Of Lime 175g 3252
Infuzions Thai SweetChili PotatoMix 110g 3242
Smiths Crnkle Chip Orgnl Big Bag 380g 3233
Thins Potato Chips Hot & Spicy 175g 3229
Kettle Sensations Camembert & Fig 150g 3219
Doritos Corn Chips Cheese Supreme 170g 3217
Pringles Barbeque 134g 3210
Doritos Corn Chip Mexican Jalapeno 150g 3204
Kettle Sweet Chilli And Sour Cream 175g 3200
Smiths Crinkle Chips Salt & Vinegar 330g 3197
Thins Chips Light& Tangy 175g 3188
Dorito Corn Chp Supreme 380g 3185
Pringles Sweet&Spcy BBQ 134g 3177
Infuzions BBQ Rib Prawn Crackers 110g 3174
Name: count, dtype: int64
PACK_SIZE
70.0 1507
90.0 3008
110.0 22387
125.0 1454
134.0 25102
135.0 3257
150.0 43131
160.0 2970
165.0 15297
170.0 19983
175.0 66390
180.0 1468
190.0 2995
200.0 4473
210.0 6272
220.0 1564
250.0 3169
270.0 6285
300.0 15166
330.0 12540
380.0 6418
Name: count, dtype: int64
```



```
In [4]: # Cell 7: Extract & clean brand name
def extract_brand(name):
    # Take first word, fix known inconsistencies
    brand = name.split()[0].upper()
    corrections = {
        'RED': 'RRD', 'DORITO': 'DORITOS', 'SNBTS': 'SUNBITES',
        'INFZNS': 'INFUZIONI', 'WW': 'WOOLWORTHS', 'SMITH': 'SMITHS',
        'NCC': 'NATURAL', 'GRAIN': 'GRNWVES'
    }
    return corrections.get(brand, brand)

transaction['BRAND'] = transaction['PROD_NAME'].apply(extract_brand)
print(transaction['BRAND'].value_counts().head(15))
```

```
BRAND
KETTLE      41288
SMITHS      31823
DORITOS     28147
PRINGLES    25102
RRD         17779
WOOLWORTHS  14757
INFUZIONI   14201
THINS       14075
COBS        9693
TOSTITOS    9471
TWISTIES    9454
OLD         9324
GRNWVES     7740
NATURAL     7469
TYRRELLS    6442
Name: count, dtype: int64
```

```
In [5]: # Cell 8: Outlier detection – suspicious large quantities
print(transaction['PROD_QTY'].describe())
print(transaction[transaction['PROD_QTY'] >= 50])      # → 200 qty outli
```

```
# Remove outlier customer (LYLTY_CARD_NBR == 226000)
transaction = transaction[transaction['LYLTY_CARD_NBR'] != 226000].copy()
print("After removal:", transaction['PROD_QTY'].describe())
```

```
count      264836.000000
mean        1.907309
std         0.643654
min         1.000000
25%         2.000000
50%         2.000000
75%         2.000000
max         200.000000
Name: PROD_QTY, dtype: float64
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	\
69762	2018-08-19	226	226000	226201	4	
69763	2019-05-20	226	226000	226210	4	

	PROD_NAME	PROD_QTY	TOT_SALES	PACK_SIZE	\
69762	Dorito Corn Chp Supreme 380g	200	650.0	380.0	
69763	Dorito Corn Chp Supreme 380g	200	650.0	380.0	

	BRAND
69762	DORITOS
69763	DORITOS

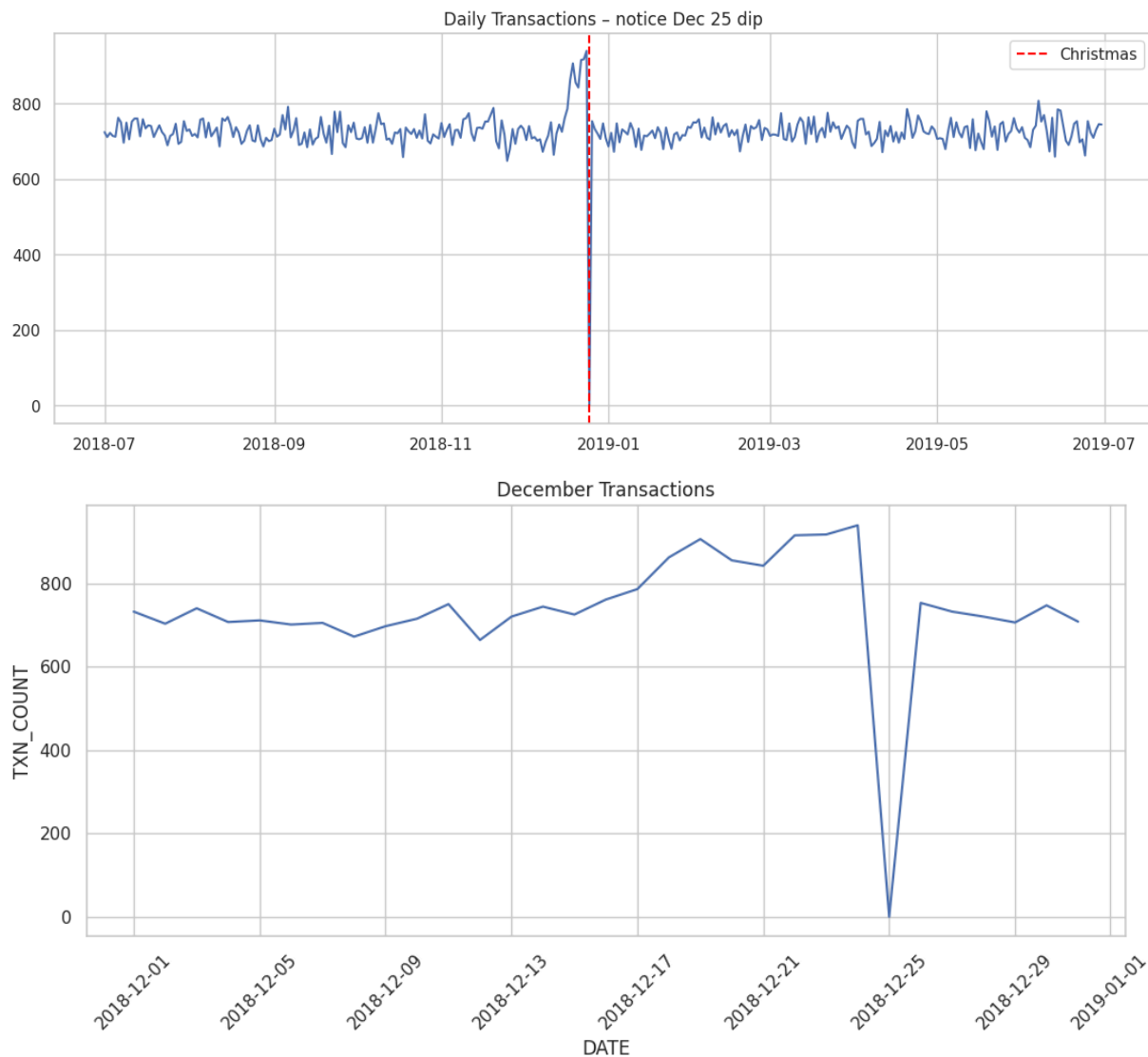
```
After removal: count      264834.000000
mean        1.905813
std         0.343436
min         1.000000
25%         2.000000
50%         2.000000
75%         2.000000
max         5.000000
Name: PROD_QTY, dtype: float64
```

```
In [6]: # Cell 9: Transactions per day + Christmas dip check
daily = transaction.groupby('DATE').size().reset_index(name='TXN_COUNT')

all_dates = pd.date_range(start='2018-07-01', end='2019-06-30')
daily_full = pd.DataFrame({'DATE': all_dates}).merge(daily, on='DATE', how='left')

plt.figure(figsize=(14,5))
plt.plot(daily_full['DATE'], daily_full['TXN_COUNT'])
plt.title("Daily Transactions – notice Dec 25 dip")
plt.axvline(pd.to_datetime('2018-12-25'), color='red', linestyle='--', label='Christmas')
plt.legend()
plt.show()

# Zoom December
dec = daily_full[daily_full['DATE'].dt.month == 12]
plt.figure(figsize=(12,5))
sns.lineplot(x='DATE', y='TXN_COUNT', data=dec)
plt.title("December Transactions")
plt.xticks(rotation=45)
plt.show()
```



```
In [7]: # Cell 10: Merge datasets
data = transaction.merge(behaviour, on='LYLTY_CARD_NBR', how='left')
print(data.isna().sum()) # should be 0
```

```
DATE                0
STORE_NBR           0
LYLTY_CARD_NBR      0
TXN_ID              0
PROD_NBR            0
PROD_NAME           0
PROD_QTY            0
TOT_SALES           0
PACK_SIZE           0
BRAND               0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

```
In [8]: # Cell 11: Key customer segment metrics

# Total sales by segment
sales_seg = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum()
```

```

plt.figure(figsize=(12,6))
sns.barplot(x='LIFESTAGE', y='TOT_SALES', hue='PREMIUM_CUSTOMER', data=sales)
plt.xticks(rotation=45, ha='right')
plt.title("Total Sales by Lifestage & Premium Segment")
plt.tight_layout()
plt.show()

# Number of unique customers
cust_seg = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR'].
sns.barplot(x='LIFESTAGE', y='CUSTOMERS', hue='PREMIUM_CUSTOMER', data=cust_seg)
plt.xticks(rotation=45, ha='right')
plt.title("Unique Customers by Segment")
plt.show()

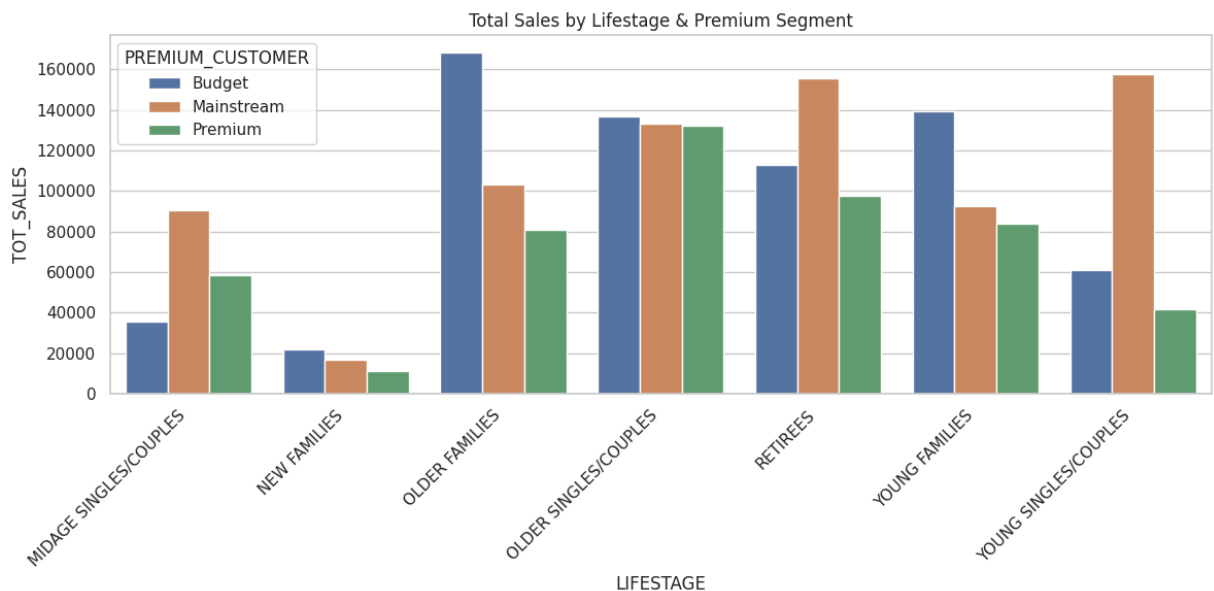
# Avg units per customer
units = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).agg(
    UNITS=('PROD_QTY', 'sum'),
    CUST=('LYLTY_CARD_NBR', 'nunique')
).reset_index()
units['AVG_UNITS'] = units['UNITS'] / units['CUST']

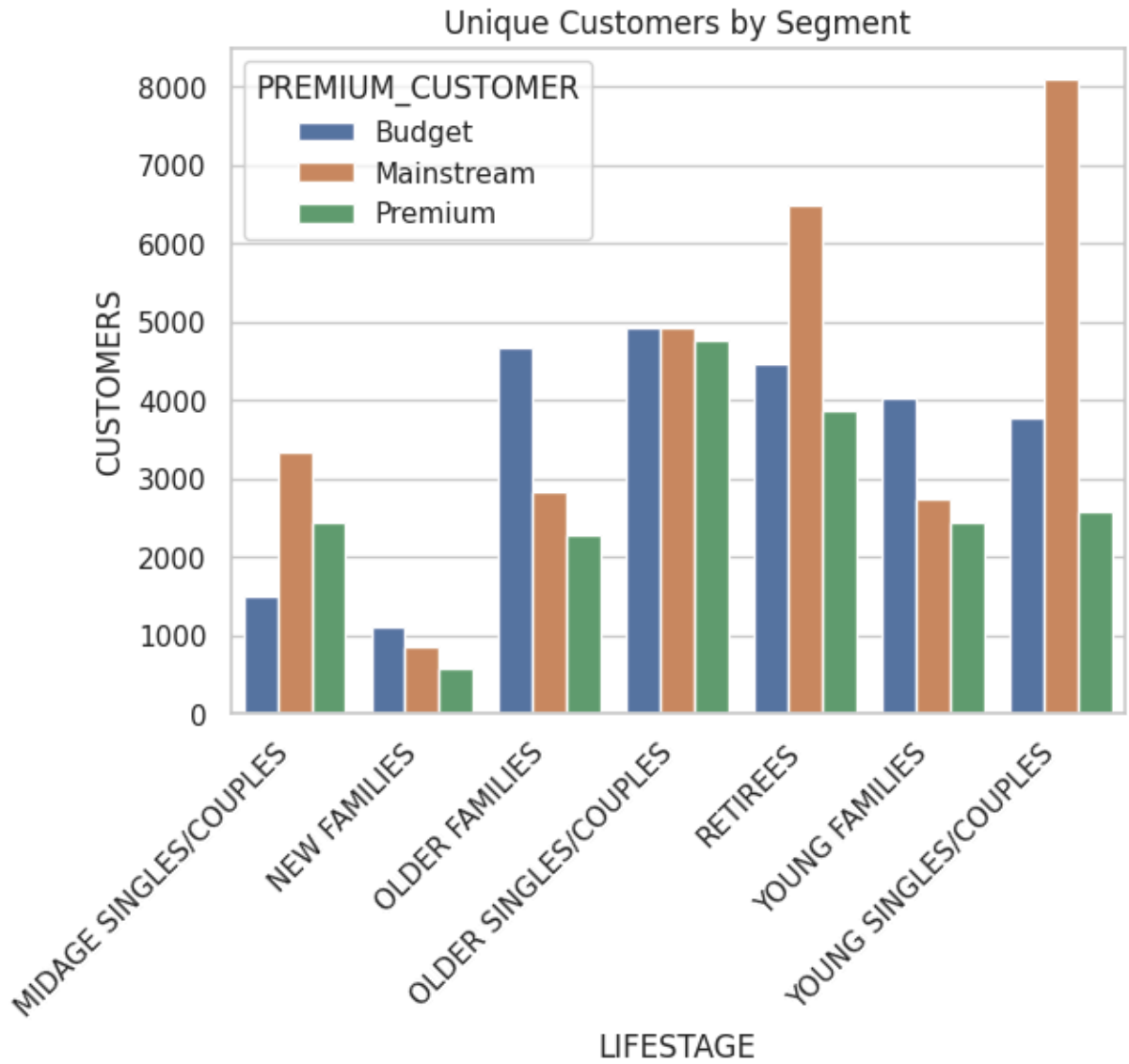
sns.barplot(x='LIFESTAGE', y='AVG_UNITS', hue='PREMIUM_CUSTOMER', data=units)
plt.xticks(rotation=45)
plt.title("Avg Units per Customer")
plt.show()

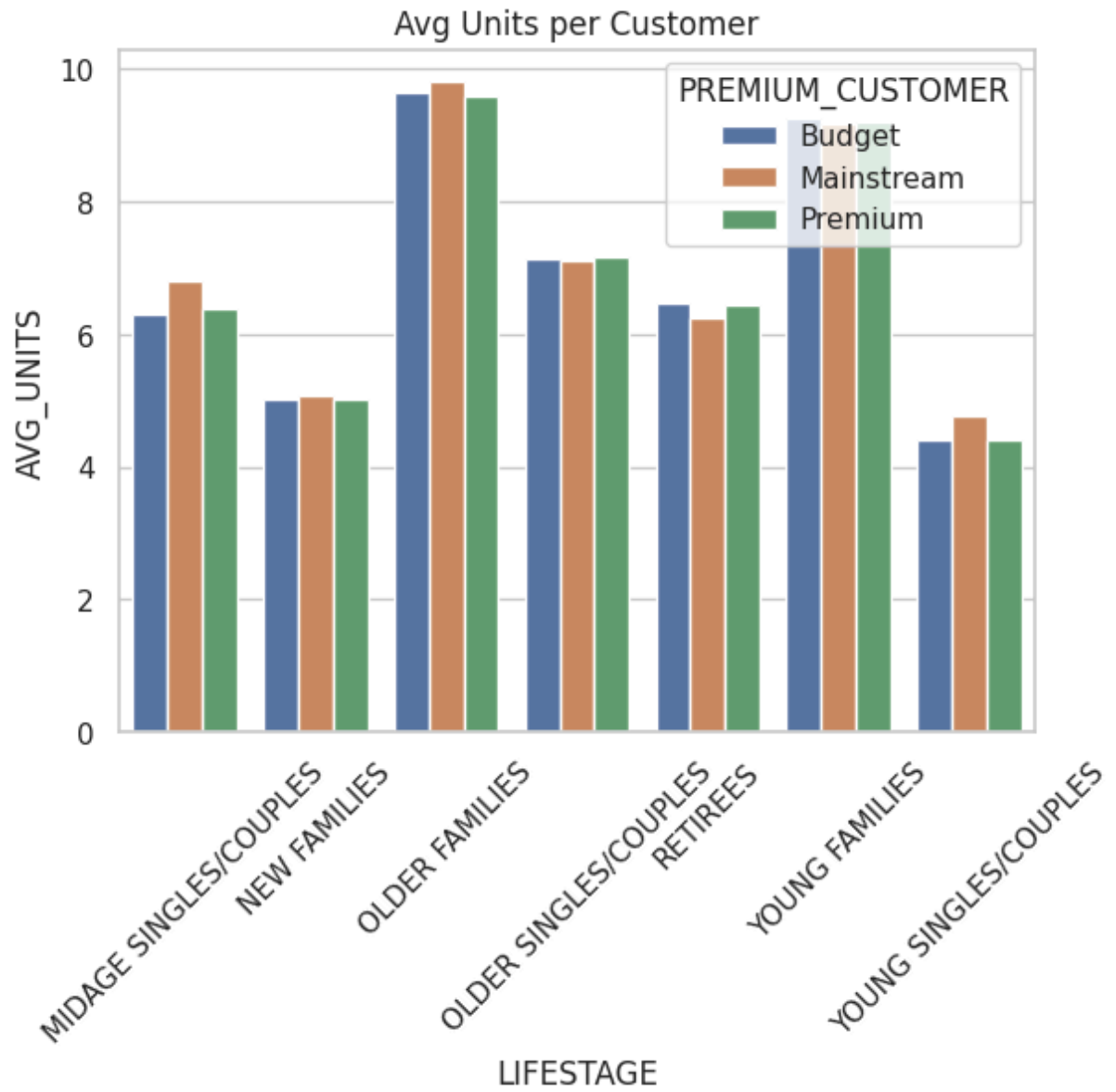
# Avg price per unit
price = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).agg(
    SALES=('TOT_SALES', 'sum'),
    QTY=('PROD_QTY', 'sum')
).reset_index()
price['AVG_PRICE'] = price['SALES'] / price['QTY']

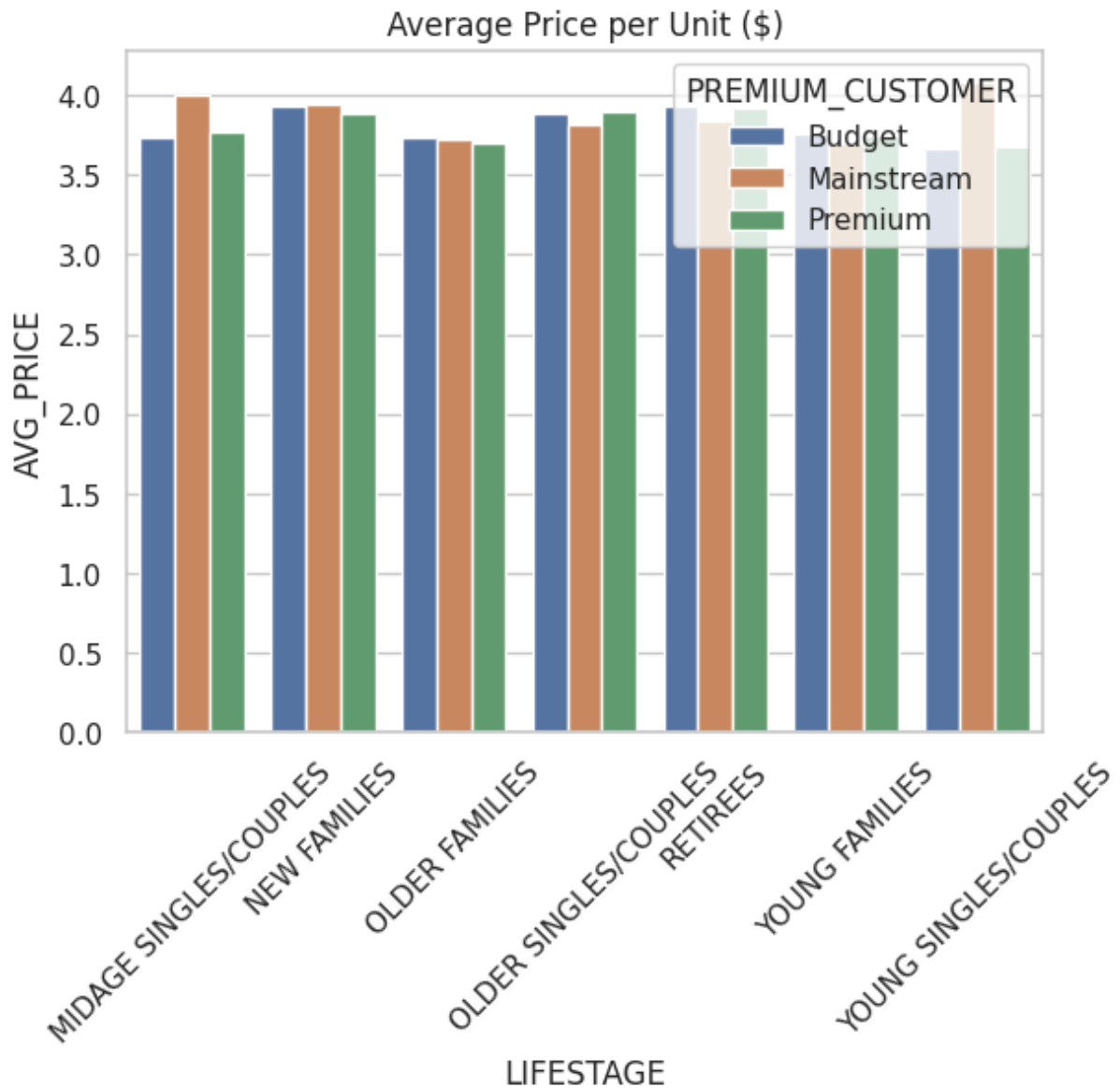
sns.barplot(x='LIFESTAGE', y='AVG_PRICE', hue='PREMIUM_CUSTOMER', data=price)
plt.xticks(rotation=45)
plt.title("Average Price per Unit ($)")
plt.show()

```









```
In [9]: # Cell 12: Statistical test – Mainstream young/mid singles/couples pay more?
from scipy.stats import ttest_ind
```

```
mainstream = data[
    data['LIFESTAGE'].isin(['YOUNG SINGLES/COUPLES', 'MIDAGE SINGLES/COUPLES']) &
    (data['PREMIUM_CUSTOMER'] == 'Mainstream')
]
premium_budget = data[
    data['LIFESTAGE'].isin(['YOUNG SINGLES/COUPLES', 'MIDAGE SINGLES/COUPLES']) &
    (data['PREMIUM_CUSTOMER'] != 'Mainstream')
]
```

```
main_price = mainstream['TOT_SALES'] / mainstream['PROD_QTY']
other_price = premium_budget['TOT_SALES'] / premium_budget['PROD_QTY']
```

```
t_stat, p_val = ttest_ind(main_price, other_price, equal_var=False)
print(f"t-test p-value: {p_val:.2e} → significantly higher price in mainstream")
```

t-test p-value: 0.00e+00 → significantly higher price in mainstream

```
In [10]: # Cell 13: Deep dive – affinity for young mainstream singles/couples
target = data[(data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (data['PREMIU
other = data[~((data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (data['PREM

brand_aff = (
    target.groupby('BRAND')['PROD_QTY'].sum() / target['PROD_QTY'].sum()
).to_frame('target_share').join(
    other.groupby('BRAND')['PROD_QTY'].sum() / other['PROD_QTY'].sum()
).rename(columns={'PROD_QTY': 'other_share'})
brand_aff['affinity'] = brand_aff['target_share'] / brand_aff['other_share']
print(brand_aff.sort_values('affinity', ascending=False).head(10))

# Same for pack size
pack_aff = (
    target.groupby('PACK_SIZE')['PROD_QTY'].sum() / target['PROD_QTY'].sum()
).to_frame('target_share').join(
    other.groupby('PACK_SIZE')['PROD_QTY'].sum() / other['PROD_QTY'].sum()
).rename(columns={'PROD_QTY': 'other_share'})
pack_aff['affinity'] = pack_aff['target_share'] / pack_aff['other_share']
print(pack_aff.sort_values('affinity', ascending=False).head(8))
```

	target_share	other_share	affinity
BRAND			
TYRRELLS	0.029587	0.023933	1.236235
TWISTIES	0.043306	0.035283	1.227401
KETTLE	0.185649	0.154216	1.203823
TOSTITOS	0.042581	0.035377	1.203638
OLD	0.041598	0.034753	1.196958
PRINGLES	0.111980	0.093743	1.194536
DORITOS	0.122877	0.105277	1.167176
COBS	0.041856	0.036375	1.150700
INFUZIONI	0.060649	0.053157	1.140947
THINS	0.056611	0.053084	1.066445
	target_share	other_share	affinity
PACK_SIZE			
270.0	0.029846	0.023377	1.276694
380.0	0.030156	0.023832	1.265361
330.0	0.057465	0.046727	1.229814
134.0	0.111980	0.093743	1.194536
110.0	0.099658	0.083642	1.191482
210.0	0.027309	0.023401	1.167002
135.0	0.013849	0.012180	1.136997
250.0	0.013460	0.011905	1.130611